

EC200U&EG91xU Series QuecOpen

RTC API Reference Manual

LTE Standard Module Series

Version: 1.1

Date: 2023-08-23

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-10-08	Neo KONG	Creation of the document
1.0	2021-10-19	Neo KONG	First official release
1.1	2023-08-23	Neo KONG/ Zack TAN	1. Added the applicable modules EG912U-GL and EG915U series. 2. Added a member QL_RTC_SET_CFG_ERR to the enumeration of ql_errcode_rtc_e (Chapter 2.3.1.1). 3. Added the functions ql_rtc_get_cfg() and ql_rtc_set_cfg(), and the structure ql_rtc_cfg_t (Chapters 2.3.9 & 2.3.10). 4. Updated the log capturing tool (Chapter 3.2).

Contents

About the Document.....	3
Contents	4
Table Index.....	5
Figure Index	6
1 Introduction	7
2 RTC API.....	8
2.1. Header File.....	8
2.2. API Overview	8
2.3. API Description	9
2.3.1. ql_rtc_set_time.....	9
2.3.1.1. ql_errcode_rtc_e	9
2.3.1.2. ql_rtc_time_t.....	10
2.3.2. ql_rtc_get_time.....	11
2.3.3. ql_rtc_print_time.....	11
2.3.4. ql_rtc_set_alarm.....	11
2.3.5. ql_rtc_get_alarm	12
2.3.6. ql_rtc_enable_alarm.....	12
2.3.7. ql_rtc_register_cb.....	13
2.3.7.1. ql_rtc_cb.....	13
2.3.8. ql_rtc_get_localtime	13
2.3.9. ql_rtc_get_cfg.....	14
2.3.9.1. ql_rtc_cfg_t.....	14
2.3.10. ql_rtc_set_cfg.....	15
3 Example	16
3.1. Development Demo	16
3.2. RTC Function Debugging	18
4 Appendix References	20

Table Index

Table 1: API Overview 8

Table 2: Related Documents..... 20

Table 3: Terms and Abbreviations 20

Figure Index

Figure 1: ql_rtc_app_init()	16
Figure 2: ql_rtc_demo_thread().....	17
Figure 3: ql_init_demo_thread()	18
Figure 4: Ports in Device Manager	19
Figure 5: RTC Log Information.....	19

1 Introduction

Quectel EC200U series and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS system. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document primarily outlines RTC API functions, RTC development demo and related debugging for the modules in QuecOpen® solution.

2 RTC API

2.1. Header File

ql_api_rtc.h, the header file of RTC API, is in the `\components\ql-kernel\inc` directory. Unless otherwise specified, all the header files mentioned in this document are in this directory.

2.2. API Overview

Table 1: API Overview

Function	Description
<i>ql_rtc_set_time()</i>	Sets the RTC time.
<i>ql_rtc_get_time()</i>	Gets the current RTC time.
<i>ql_rtc_print_time()</i>	Displays the RTC time.
<i>ql_rtc_set_alarm()</i>	Sets the alarm time.
<i>ql_rtc_get_alarm()</i>	Gets the alarm time.
<i>ql_rtc_enable_alarm()</i>	Enables the alarm feature.
<i>ql_rtc_register_cb()</i>	Registers alarm callback function.
<i>ql_rtc_get_localtime()</i>	Gets the local time (including time zone).
<i>ql_rtc_get_cfg()</i>	Gets the RTC startup configuration.
<i>ql_rtc_set_cfg()</i>	Sets the RTC startup configuration.

2.3. API Description

2.3.1. ql_rtc_set_time

This function sets the RTC time.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_set_time(ql_rtc_time_t *tm)
```

- **Parameter**

tm:

[In] RTC time structure pointer. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.1.1. ql_errcode_rtc_e

The enumeration of error codes of RTC API:

```
typedef enum
{
    QL_RTC_SUCCESS = QL_SUCCESS,
    QL_RTC_INVALID_PARAM_ERR = 1|QL_RTC_ERRCODE_BASE,
    QL_RTC_SET_TIME_ERROR,
    QL_RTC_SET_CB_ERR,
    QL_RTC_ENABLE_ALARM_ERR,
    QL_RTC_REMOVE_ALARM_ERR,
    QL_RTC_SET_CFG_ERR
}ql_errcode_rtc_e
```

- **Member**

Member	Description
QL_RTC_SUCCESS	Successful execution
QL_RTC_INVALID_PARAM_ERR	Invalid parameter
QL_RTC_SET_TIME_ERROR	RTC time setting error
QL_RTC_SET_CB_ERR	Callback function registration error

<i>QL_RTC_ENABLE_ALARM_ERR</i>	Enabling alarm error
<i>QL_RTC_REMOVE_ALARM_ERR</i>	Removing alarm error
<i>QL_RTC_SET_CFG_ERR</i>	Incorrect RTC startup configuration

2.3.1.2. ql_rtc_time_t

The structure of RTC time:

```
typedef struct ql_rtc_time_struct {
    int tm_sec;
    int tm_min;
    int tm_hour;
    int tm_mday;
    int tm_mon;
    int tm_year;
    int tm_wday;
}ql_rtc_time_t
```

● Parameter

Type	Parameter	Description
int	<i>tm_sec</i>	Second. Range: 0–59.
int	<i>tm_min</i>	Minute. Range: 0–59.
int	<i>tm_hour</i>	Hour. Range: 0–23.
int	<i>tm_mday</i>	Day. Range: 1–31.
int	<i>tm_mon</i>	Month. Range: 1–12.
int	<i>tm_year</i>	Year. Range: 2000–2100.
int	<i>tm_wday</i>	Day of the week. Range: 0–6. 0 represents Sunday.

2.3.2. ql_rtc_get_time

This function gets the current RTC time.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_get_time(ql_rtc_time_t *tm)
```

- **Parameter**

tm:

[Out] RTC time structure pointer. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.3. ql_rtc_print_time

This function displays the RTC time.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_print_time(ql_rtc_time_t tm)
```

- **Parameter**

tm:

[In] RTC time structure. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.4. ql_rtc_set_alarm

This function sets the alarm time.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_set_alarm (ql_rtc_time_t* tm)
```

- **Parameter**

tm:

[In] RTC time structure pointer. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.5. ql_rtc_get_alarm

This function gets the alarm time.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_get_alarm (ql_rtc_time_t* tm)
```

- **Parameter**

tm:

[Out] RTC time structure pointer. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.6. ql_rtc_enable_alarm

This function enables the alarm feature.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_enable_alarm (unsigned char on_off)
```

- **Parameter**

on_off:

[In] Indicates whether to enable or remove the alarm.

- 1 Enable the alarm
- 0 Remove the alarm

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.7. ql_rtc_register_cb

This function registers alarm callback function.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_register_cb (ql_rtc_cb cb)
```

- **Parameter**

cb:

[In] Alarm callback function pointer. See **Chapter 2.3.7.1** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.7.1. ql_rtc_cb

The function is alarm callback function.

- **Prototype**

```
typedef void (*ql_rtc_cb)(void)
```

- **Parameter**

None

- **Return Value**

None

2.3.8. ql_rtc_get_localtime

This function gets the local time (including time zone).

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_get_localtime(ql_rtc_time_t *tm)
```

- **Parameter**

tm:

[Out] RTC time structure pointer. See **Chapter 2.3.1.2** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.9. ql_rtc_get_cfg

This function gets the RTC startup configuration.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_get_cfg(ql_rtc_cfg_t *ql_rtc_cfg)
```

- **Parameter**

ql_rtc_cfg:

[Out] RTC startup configuration structure. See **Chapter 2.3.9.1** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

2.3.9.1. ql_rtc_cfg_t

The structure of RTC startup configuration:

```
typedef struct ql_rtc_cfg_struct {
    quec_enable_e nv_cfg;
    quec_enable_e rtc_cfg;
    quec_enable_e nwt_cfg;
}ql_rtc_cfg_t
```

- **Parameter**

Type	Parameter	Description
quec_enable_e	<i>nv_cfg</i>	Specifies whether to read the time saved in NV upon startup as the initial RTC value.
quec_enable_e	<i>rtc_cfg</i>	Specifies whether to read the RTC register time upon startup as the initial RTC value.
quec_enable_e	<i>nwt_cfg</i>	Specifies whether to synchronize the base station

time to RTC time after connecting the base station.

2.3.10. ql_rtc_set_cfg

This function sets the RTC startup configuration.

- **Prototype**

```
ql_errcode_rtc_e ql_rtc_set_cfg(ql_rtc_cfg_t *ql_rtc_cfg)
```

- **Parameter**

ql_rtc_cfg:

[In] RTC startup configuration structure. See **Chapter 2.3.9.1** for details.

- **Return Value**

See **Chapter 2.3.1.1** for error codes.

3 Example

This chapter outlines how to use the above APIs for RTC development and related debugging on application side.

3.1. Development Demo

QuecOpen SDK provides the RTC API demo file *ql_rtc_demo.c*, which is in the `\components\ql-application\rtc` directory. The file includes functions dedicated to acquiring, setting, displaying time information, and setting alarm. The entry function *ql_rtc_app_init()*, as shown in the figure below, is used for initiating *ql_rtc_demo_thread()* task.

```
void ql_rtc_app_init()
{
    QIOSStatus err = QL_OSI_SUCCESS;
    ql_task_t rtc_task = NULL;

    err = ql_rtos_task_create(&rtc_task, 1024, APP_PRIORITY_NORMAL, "ql_rtc_demo", ql_rtc_demo_thread, NULL, 1);
    if( err != QL_OSI_SUCCESS )
    {
        QL_RTCDEMO_LOG("rtc demo task created failed");
    }
}
```

Figure 1: *ql_rtc_app_init()*

This demo sequentially executes the following functions: getting and displaying the current time, setting a new time, retrieving and displaying the updated time, and setting and enabling the alarm function. When the alarm time arrives, *ql_rtc_test_callback()* will be called automatically.

```

static void ql_rtc_demo_thread()
{
    QL_RTCDEMO_LOG("ql rtc demo enter! \n");

    ql_rtc_time_t tm;

    //get current time
    ql_rtc_get_time(&tm);
    ql_rtc_print_time(tm);

    //2020-12-22 16:50:30 [WED]
    tm.tm_year = 2020;
    tm.tm_mon = 12;
    tm.tm_mday = 22;
    tm.tm_wday = 2;
    tm.tm_hour = 16;
    tm.tm_min = 50;
    tm.tm_sec = 30;
    ql_rtc_print_time(tm);

    //set time
    ql_rtc_set_time(&tm);

    ql_rtc_get_time(&tm);
    ql_rtc_print_time(tm);

    //set & enable alarm
    >> tm.tm_sec += 20;
    >> ql_rtc_set_alarm(&tm);
    >> ql_rtc_register_cb(ql_rtc_test_callback);
    >> ql_rtc_enable_alarm(1);

    >> while (1)
    >> {
    >>     ql_rtc_get_time(&tm);
    >>     QL_RTCDEMO_LOG("=====print current time=====\\r\\n");
    >>     ql_rtc_print_time(tm);
    >>     ql_rtc_task_sleep_s(5);
    >> }
} « end ql_rtc_demo_thread »
    
```

Figure 2: ql_rtc_demo_thread()

As shown in the following figure, the above demo will not start by default in `ql_init_demo_thread()`. Please uncomment `ql_rtc_app_init()` when testing is required.

```
#ifdef QL_APP_FEATURE_GNSS
    //ql_gnss_app_init();
#endif

#ifdef QL_APP_FEATURE_SPI
    //ql_spi_demo_init();
#endif

#ifdef QL_APP_FEATURE_CAMERA
    //ql_camera_app_init();
#endif

#ifdef QL_APP_FEATURE_SPI_FLASH
    //ql_spi_flash_demo_init();
#endif

#if defined (QL_APP_FEATURE_WIFISCAN)
    //ql_wifiscan_app_init();
#endif

#ifdef QL_APP_FEATURE_HTTP_FOTA
    //ql_fota_http_app_init();
#endif
#ifdef QL_APP_FEATURE_DECODER
    //ql_decoder_app_init();
#endif

#ifdef QL_APP_FEATURE_KEYPAD
    //ql_keypad_app_init();
#endif

#ifdef QL_APP_FEATURE_RTC
    //ql_rtc_app_init();
#endif
```



Figure 3: `ql_init_demo_thread()`

3.2. RTC Function Debugging

RTC function of the modules can be debugged on Quectel LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to the PC with a USB cable. USB AP Log Port is the primary interface for displaying the system debugging information. You can view information of the example through USB AP Log Port after capturing the log with *cooltools*. For details about log capture, see [document \[2\]](#).

-  Quectel USB AP Log Port (COM93)
-  Quectel USB AT Port (COM92)
-  Quectel USB CP Log Port (COM95)
-  Quectel USB Diag Port (COM91)
-  Quectel USB MOS Port (COM94)
-  Quectel USB NMEA Port (COM113)

Figure 4: Ports in Device Manager

ql_rtc_app_init() is called automatically after the module startup. The debugging information output from the AP Log Port is shown in the following figure:

Index	Received	Tick	Level	
257	11:11:18.351	15298	RTCA/I	RTC store nv length/6
819	11:11:21.558	828	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 64] ql rtc demo enter!
820	11:11:21.558	830	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 17:03:18 [TUS]
821	11:11:21.558	830	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:30 [TUS]
822	11:11:21.558	831	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:30 [TUS]
826	11:11:21.558	836	RTCA/I	RTC set alarm day/7661 hour/16 min/50 sec/50
827	11:11:21.558	861	RTCA/I	RTC store nv length/70
828	11:11:21.558	866	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
829	11:11:21.581	867	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:30 [TUS]
832	11:11:21.649	2939	RTCA/D	RTC intr 0x00001000
841	11:11:26.445	17105	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
842	11:11:26.504	17105	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:34 [TUS]
1037	11:11:31.436	33490	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
1038	11:11:31.488	33490	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:39 [TUS]
1046	11:11:36.444	49873	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
1047	11:11:36.503	49873	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:44 [TUS]
1094	11:11:41.449	721	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
1095	11:11:41.449	722	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:49 [TUS]
1096	11:11:41.449	838	QOPN/I	[ql_RTCDEMO][ql_rtc_test_callback, 49] ql rtc test callback come in!
1097	11:11:41.449	839	QOPN/I	[ql_RTCDEMO][ql_rtc_test_callback, 57] =====callback print alarm time=====
1098	11:11:41.449	839	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:50 [TUS]
1099	11:11:41.449	839	RTCA/I	RTC unset alarm
1100	11:11:41.505	862	RTCA/I	RTC store nv length/6
1124	11:11:46.440	17106	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
1125	11:11:46.501	17106	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:54 [TUS]

Figure 5: RTC Log Information

4 Appendix References

Table 2: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen_CSDK_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen_Log_Capture_Guide

Table 3: Terms and Abbreviations

Abbreviation	Description
AP	Application Processor
API	Application Programming Interface
EVB	Evaluation Board
IoT	Internet of Things
LTE	Long-Term Evolution
RTC	Real-Time Clock
RTOS	Real-Time Operating System
SDK	Software Development Kit
USB	Universal Serial Bus